

# Academic References and Further Reading

EE5203 Microprocessors — where the full story is

The lectures in this course introduce each concept **conceptually**: enough mechanism to configure, use, and debug it, without the full architectural detail. This page maps every simplified concept to the authoritative document where the complete story lives, so that a curious student — or a future project — can go one level deeper with a known-good source instead of a random blog.

Each lecture gets its own section. Entries are added as lectures are revised.

## If a link stops working

Links below point at the current official locations (verified 2026-07-17). If one rots:

- **ST documents** (RM..., PM..., UM..., MB... numbers): search the document number at [st.com](https://www.st.com); the number uniquely identifies the document and survives site reorganizations.
- **Arm documents** (DDI... numbers): search at [developer.arm.com](https://developer.arm.com).
- **Books**: available through the university library or the cited publisher; Lee & Seshia is free and legal at [leeseshia.org](https://leeseshia.org).

## Primary documents used throughout the course

| Short name             | Full citation                                                                                                                                                                       | What it is for                                                                                                        |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <a href="#">RM0368</a> | STMicroelectronics,<br><i>RM0368:</i><br><i>STM32F401xB/C and</i><br><i>STM32F401xD/E</i><br><i>advanced Arm®-based</i><br><i>32-bit MCUs —</i><br><i>Reference Manual</i><br>(PDF) | The register-level truth<br>for every peripheral on<br>our chip: GPIO,<br>SYSCFG, EXTI, timers,<br>ADC, I2C, SPI, DMA |

| Short name                       | Full citation                                                                                                                                                | What it is for                                                                                         |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <a href="#">PM0214</a>           | STMicroelectronics, <i>PM0214: STM32 Cortex®-M4 MCUs and MPUs programming manual</i> (PDF)                                                                   | The processor core seen from software: instruction set, exception model, NVIC, SysTick, core registers |
| <a href="#">UM1724</a>           | STMicroelectronics, <i>UM1724: STM32 Nucleo-64 boards (MB1136) — User Manual</i> (PDF)                                                                       | The board itself: B1 and LD2 circuits, jumpers, ST-LINK, power, <b>board revision differences</b>      |
| <a href="#">MB1136 schematic</a> | STMicroelectronics, <i>MB1136 schematic pack</i> , rev C-04 (PDF)                                                                                            | The actual circuit diagram of the Nucleo-F401RE — the version used in lecture                          |
| <a href="#">UM1725</a>           | STMicroelectronics, <i>UM1725: Description of STM32F4 HAL and low-layer drivers</i> (PDF)                                                                    | Signature and behavior of every <code>HAL_...</code> function the labs use                             |
| <a href="#">Yiu 2014</a>         | J. Yiu, <i>The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors</i> , 3rd ed., Newnes, 2014. ISBN 978-0-12-408082-9                             | The standard book-length treatment of the core: pipeline, exceptions, NVIC, low-power, debug           |
| <a href="#">DDI 0403</a>         | Arm Ltd., <i>ARMv7-M Architecture Reference Manual</i> (Arm DDI 0403)                                                                                        | The formal architecture specification — for when even Yiu is not precise enough                        |
| <a href="#">Lee &amp; Seshia</a> | E. A. Lee and S. A. Seshia, <i>Introduction to Embedded Systems: A Cyber-Physical Systems Approach</i> , 2nd ed., MIT Press, 2017 — <a href="#">free PDF</a> | The academic textbook view: interrupts, concurrency, real-time reasoning                               |
| <a href="#">White 2024</a>       | E. White, <i>Making Embedded Systems</i> , 2nd ed., O'Reilly, 2024                                                                                           | Practical engineering judgment: interrupt hygiene, debouncing, system design patterns                  |
| <a href="#">C11</a>              | ISO/IEC 9899:2011, <i>Programming languages — C</i> (link is N1570, the final public draft, PDF)                                                             | The C language standard; final authority on <code>volatile</code> , integer types, undefined behavior  |

## L01 — Embedded Systems, the C Toolchain, and Debugging

| Concept as taught                                                                                                                                                                                                                                                                                                                                 | Where the full story is                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What an embedded system is</b> — dedicated function + constraints<br><b>The Nucleo-F401RE board</b> — ST-LINK, LD2, B1, headers<br><b>Processor / MCU / system model</b><br><b>The toolchain</b> — compile, link, flash, debug<br><br><b>First C statements</b> — <code>main()</code> , declarations, <code>if</code> , <code>while</code> (1) | <a href="#">Lee &amp; Seshia</a> , ch. 1 <i>Introduction</i> (free PDF); <a href="#">White 2024</a> , ch. 1<br><a href="#">UM1724 Rev 17</a> , §6 <i>Hardware layout</i> — p. 13; <a href="#">MB1136 C-04 schematic</a><br><a href="#">PM0214 Rev 10</a> , §2.1 <i>Programmers model</i> — p. 17; Yiu 2014, ch. 1–2<br>Yiu 2014, ch. 2 (development flow and what CMSIS standardizes); the VS Code build terminal of any generated project shows each stage’s real output<br><a href="#">C11 (N1570)</a> , §6.8.4 <i>Selection statements</i> — p. 166; program startup is §5.1.2.2.1 |

## L02 — Values, Decisions, Loops, and Arrays

| Concept as taught                                                                                                                                                                                         | Where the full story is                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>if / else if / else</b><br><br><b>while and for loops</b><br><br><b>Arrays and indexing from zero</b><br><br><b>uint16_t and friends</b><br><br><b>Integer division</b> — <code>a / b</code> truncates | <a href="#">C11 (N1570)</a> , §6.8.4 <i>Selection statements</i> — p. 166<br><a href="#">C11 (N1570)</a> , §6.8.5 <i>Iteration statements</i> — p. 168<br><a href="#">C11 (N1570)</a> , §6.5.2.1 <i>Array subscripting</i> — p. 98 — out-of-range access is undefined behavior, which is why <code>samples[8]</code> is not “just the next value”<br><a href="#">C11 (N1570)</a> , §7.20 <i>Integer types</i> <code>&lt;stdint.h&gt;</code> — p. 307<br><a href="#">C11 §6.5.5</a> (multiplicative operators): division truncates toward zero |

## L03 — Functions, Arrays, Strings, and Multiple C Files

| Concept as taught                       | Where the full story is                                                   |
|-----------------------------------------|---------------------------------------------------------------------------|
| <b>Declarations, definitions, calls</b> | <a href="#">C11 (N1570)</a> , §6.9.1 <i>Function definitions</i> — p. 174 |

| Concept as taught                                                   | Where the full story is                                                                                                                                     |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pass by value</b> — parameters are copies                        | C11 (N1570), §6.5.2.2 <i>Function calls</i> — p. 99                                                                                                         |
| <b>Headers, multiple files, the linker</b>                          | C11 (N1570), §6.2.2 <i>Linkages of identifiers</i> — p. 54; <code>#include</code> / <code>#define</code> are §6.10 <i>Preprocessing directives</i> — p. 184 |
| <b>Why array parameters need a count</b> — arrays decay to pointers | C11 §6.7.6.3¶7; the L04 pointer material makes this concrete                                                                                                |
| <b>C strings and <code>\0</code>; <code>strcmp</code></b>           | C11 (N1570), §7.24 <i>String handling</i> <code>&lt;string.h&gt;</code> — p. 380                                                                            |

## L04 — Pointers, Structures, Bits, Registers, and GPIO

The lecture builds `GPIOA->BSRR = (1U << 5);` from its parts: addresses, pointers, structures, masks, and the GPIO registers.

| Concept as taught                                                                    | Where the full story is                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Addresses and memory</b> — “memory is a numbered collection of storage locations” | PM0214 Rev 10, §2.2 <i>Memory model</i> — p. 28: the real Cortex-M4 memory map, including where peripherals live                                                                                                                                               |
| <b>Pointers: <code>&amp;</code> and <code>*</code></b>                               | C11 (N1570), §6.5.3.2 <i>Address and indirection operators</i> — p. 106                                                                                                                                                                                        |
| <b>Structures and <code>.</code> / <code>-&gt;</code></b>                            | C11 (N1570), §6.7.2.1 <i>Structure and union specifiers</i> — p. 130                                                                                                                                                                                           |
| <b>Bitwise operators, masks, the U suffix</b>                                        | C11 (N1570), §6.5.7 <i>Bitwise shift operators</i> — p. 112, then §6.5.10–6.5.12 for <code>&amp;</code> , <code>^</code> , <code> </code> ; §6.5.7 also explains why shifting <b>signed</b> values can be undefined behavior — the reason the U suffix matters |
| <b>CMSIS register names</b> — “the device header supplies the definition”            | The actual <code>stm32f401xe.h</code> device header ( <code>GPIO_TypeDef</code> , <code>GPIOA_BASE</code> ); Yiu 2014, ch. 2 on what CMSIS standardizes                                                                                                        |
| <b>volatile register members</b>                                                     | C11 (N1570), §6.7.3 — p. 140; M. Barr, <i>How to Use C’s volatile Keyword</i>                                                                                                                                                                                  |
| <b>Peripheral clock gating</b> — “enable the clock first”                            | RM0368 Rev 6, <code>RCC_AHB1ENR</code> — p. 118; the RCC chapter (§6) explains <i>why</i> clocks are gated: power                                                                                                                                              |

| Concept as taught              | Where the full story is                                                                                                                                                                                  |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GPIO block, MODER, BSRR</b> | <a href="#">RM0368 Rev 6, §8 <i>GPIO</i> — p. 146</a> ; registers: <a href="#">GPIOx_MODER — p. 158</a> , <a href="#">GPIOx_BSRR — p. 161</a> — including what the lecture skips: OTYPER, OSPEEDR, PUPDR |

## L05 — GPIO and Hardware Abstraction

The lecture connects every HAL name to the register behavior taught in L04: same hardware, second interface.

| Concept as taught                                                                                                                        | Where the full story is                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The HAL GPIO API</b> —<br>GPIO_InitTypeDef, HAL_GPIO_Init,<br>WritePin / TogglePin / ReadPin<br><b>What the HAL actually executes</b> | <a href="#">UM1725 Rev 8, GPIO functions — p. 447</a><br><br>The <a href="#">source of stm32f4xx_hal_gpio.c</a> —<br>HAL_GPIO_WritePin is a single BSRR write                        |
| <b>HAL names → register bits</b> —<br>GPIO_MODE_OUTPUT_PP, GPIO_NOPULL,<br>speed                                                         | <a href="#">RM0368 Rev 6, §8 <i>GPIO</i> — p. 146</a> ; registers: <a href="#">GPIOx_MODER — p. 158</a> , <a href="#">GPIOx_PUPDR — p. 159</a> , <a href="#">GPIOx_BSRR — p. 161</a> |
| <b>Floating inputs, pull-up / pull-down</b>                                                                                              | <a href="#">RM0368 §8.3 (I/O port control)</a> ; the electronics context in any circuits text — the lecture’s claim is only that an undriven CMOS input is undefined                 |
| <b>B1 circuit — external pull-up, active-low</b>                                                                                         | <a href="#">MB1136 schematic rev C-04</a> ; <a href="#">UM1724 Rev 17, push-buttons — p. 24</a> ; confirmed against the <a href="#">ST Nucleo BSP source</a>                         |
| <b>Why abstraction layers exist — and what they cost</b>                                                                                 | <a href="#">White 2024</a> , ch. 2 (creating system diagrams / layering); <a href="#">Yiu 2014</a> , ch. 2 (what CMSIS standardizes and why)                                         |
| <b>Polling and its limits</b> — “the loop asks again and again”                                                                          | Forward pointer: the L06 row <i>Polling vs. interrupts</i> above                                                                                                                     |

## L06 — GPIO Interrupts

The lecture walks one button press through five plain-language stages (PRESS → NOTICE → ASK → LOOK UP → RUN YOUR CODE). Each simplification below names the document that removes it.

Page-anchored links ([#page=N](#)) below open the PDF at the cited page in the browser viewer. Page numbers were verified against the document revisions named in each entry (2026-07-17); if ST publishes a new revision, the section numbers in the text remain the reliable pointer.

| Concept as taught                                                                                                                             | Where the full story is                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Polling vs. interrupts</b> — “polling asks again and again”                                                                                | <a href="#">Lee &amp; Seshia</a> , ch. 10 <i>Input and Output</i> (interrupt mechanisms, latency, and when polling is actually the better design); Yiu 2014, §4.5                                                                                         |
| <b>What really happens between ASK and RUN YOUR CODE</b> — register stacking, vector fetch, exception entry/return (we skipped this entirely) | <a href="#">PM0214 Rev 10, §2.3 <i>Exception model</i></a> — p. 37; Yiu 2014, ch. 8 (including the 12-cycle entry sequence and tail-chaining); <a href="#">DDI 0403</a> , §B1.5 for the formal definition                                                 |
| <b>EXTI</b> — “a set of hardware edge detectors”                                                                                              | <a href="#">RM0368 Rev 6, §10 <i>EXTI</i></a> — p. 205: block diagram, IMR/EMR/RTSR/FTSR/SWIER/PR registers, the event-vs-interrupt distinction we did not cover                                                                                          |
| <b>SYSCFG routing</b> — “a four-bit field selects the port”                                                                                   | <a href="#">RM0368 Rev 6, SYSCFG_EXTICR4</a> — p. 143                                                                                                                                                                                                     |
| <b>Pending bit and write-1-to-clear</b>                                                                                                       | <a href="#">RM0368 Rev 6, EXTI_PR</a> — p. 211; the same <code>rc_w1</code> convention appears in many STM32 peripherals                                                                                                                                  |
| <b>NVIC</b> — “enable, priority, deliver”                                                                                                     | <a href="#">PM0214 Rev 10, §4.2 <i>NVIC</i></a> — p. 208; Yiu 2014, ch. 7 — including what we postponed: priority levels, preemption, sub-priority, nesting                                                                                               |
| <b>Vector table and the startup file</b>                                                                                                      | <a href="#">PM0214 Rev 10, §2.3.4 <i>Vector table</i></a> — p. 40; Yiu 2014, §4.4 and §8.5; the generated <code>startup_stm32f401retx.s</code> in every CubeMX-generated project is the concrete instance                                                 |
| <b>EXTI15_10_IRQHandler / HAL helper / callback chain</b>                                                                                     | <a href="#">UM1725 Rev 8, GPIO functions</a> — p. 447 ( <code>HAL_GPIO_EXTI_IRQHandler</code> , <code>HAL_GPIO_EXTI_Callback</code> ); the ~10-line <a href="#">source of the helper</a> in <code>stm32f4xx_hal_gpio.c</code> is worth reading in the lab |

| Concept as taught                                        | Where the full story is                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>volatile</b> — “read the stored value each time”      | <a href="#">C11 (N1570), §6.7.3 — p. 140</a> ; M. Barr, <i>How to Use C’s volatile Keyword</i> , Barr Group. Note what the lecture only hints at: <b>volatile</b> does <b>not</b> make read-modify-write atomic — see <a href="#">Lee &amp; Seshia</a> , ch. 11 on shared-variable concurrency |
| <b>ISR main() shared-variable pattern</b> (button_event) | <a href="#">Lee &amp; Seshia</a> , ch. 11 (race conditions, atomicity); <a href="#">White 2024</a> , interrupt chapter (keep-ISRs-short discipline and its reasons)                                                                                                                            |
| <b>Switch bounce and the 80 ms guard</b>                 | J. Ganssle, <i>A Guide to Debouncing</i> , 2nd ed., The Ganssle Group, 2008: measured bounce data for real switches (the “settles within ~10 ms” claim comes from here), plus hardware and software debounce alternatives                                                                      |
| <b>B1 circuit — external pull-up, active-low</b>         | <a href="#">MB1136 schematic rev C-04</a> ; <a href="#">UM1724 Rev 17, push-buttons — p. 24</a> — <b>check the board revision</b> : later MB1136 revisions changed the B1 circuit, which affects the pull configuration the lab assumes                                                        |

## L07 — Timers

The lecture builds one mental model — clock → PSC → CNT → ARR → update event — and consumes it by polling, by interrupt, and as a 1 ms tick.

| Concept as taught                                                        | Where the full story is                                                                                       |
|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>The time base</b> — prescaler, counter, auto-reload, update event     | <a href="#">RM0368 Rev 6, §13 <i>TIM2 to TIM5</i> — p. 316</a> ; registers ( <a href="#">§13.4 — p. 353</a> ) |
| <b>Where the 16 MHz comes from</b> — clock tree, APB timer-clock ×2 rule | <a href="#">RM0368 Rev 6, §6.2 <i>Clocks</i> — p. 94</a>                                                      |
| <b>HAL TIM base API</b> — handle, Base_Init, Base_Start_IT, callback     | <a href="#">UM1725 Rev 8, TIM functions — p. 1065</a>                                                         |
| <b>The interrupt path</b> — reused from L06                              | The L06 rows above; the only new element is the request name <code>TIM2_IRQn</code>                           |
| <b>Tick-based scheduling and why blocking delays hurt</b>                | <a href="#">Lee &amp; Seshia</a> , ch. 10–11; <a href="#">White 2024</a> , timer and main-loop chapters       |

| Concept as taught                                                                    | Where the full story is                                           |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>SysTick</b> — the Cortex-M core timer behind <code>HAL_Delay</code> (not covered) | <a href="#">PM0214 Rev 10, §4.5 <i>SysTick</i> timer</a> — p. 246 |

## L08 — Pulse-Width Modulation

The lecture adds the capture/compare channel to L07's time base and uses it in the compare direction: `ARR` owns the frequency, `CCR` owns the duty.

| Concept as taught                                                    | Where the full story is                                                                                                                                                           |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PWM mode</b> — pin high while <code>CNT</code> < <code>CCR</code> | <a href="#">RM0368 Rev 6, §13.3.9 <i>PWM mode</i></a> — p. 336, including what the lecture skips: center-aligned mode, PWM mode 2                                                 |
| <b>Channel registers</b>                                             | <a href="#">TIMx_CCMR1</a> — p. 362, <a href="#">TIMx_CCER</a> — p. 366, <a href="#">TIMx_CCR1</a> — p. 367                                                                       |
| <b>Handing the pin to the timer</b> — alternate function             | <a href="#">RM0368 Rev 6, GPIOx_AFRL</a> — p. 162; the pin-to-AF table is the STM32F401RE datasheet (DS10086), <i>Alternate function mapping</i> — search the DS number at st.com |
| <b>HAL PWM API</b>                                                   | <a href="#">UM1725 Rev 8, TIM functions</a> — p. 1065                                                                                                                             |
| <b>The servo pulse contract</b> — 50 Hz frame, 1–2 ms pulse          | Hobby-servo convention, not an ST document: any RC-servo datasheet (e.g., SG90, S3003) states the same 20 ms / 1–2 ms numbers                                                     |
| <b>Why the average works</b> — PWM into slow loads                   | <a href="#">White 2024</a> , output/actuator chapters; the RC-filter demo linked from the deck shows it interactively                                                             |

## L09 — Input Capture

The same channel, reversed: an edge on the pin captures the free-running counter into `CCR` — a hardware timestamp.

| Concept as taught                                                         | Where the full story is                                                  |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Input capture</b> — edge copies <code>CNT</code> into <code>CCR</code> | <a href="#">RM0368 Rev 6, §13.3.5 <i>Input capture mode</i></a> — p. 333 |



| Concept as taught                                         | Where the full story is                                                                                                                                 |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Automated duty measurement</b>                         | <a href="#">RM0368 Rev 6, §13.3.6 <i>PWM input mode</i></a> — p. 334 — the hardware version of the paired-channel trick, at the cost of the whole timer |
| <b>Direct vs indirect inputs, the filter, overcapture</b> | <a href="#">TIMx_CCMR1</a> — p. 362 (CCxS, ICF), <a href="#">TIMx_CCER</a> — p. 366; overcapture flags in the TIMx_SR description                       |
| <b>HAL capture API</b>                                    | <a href="#">UM1725 Rev 8, TIM functions</a> — p. 1065; <a href="#">HAL_TIM_ReadCapturedValue</a> — p. 1097                                              |
| <b>Ultrasonic ranging</b> — echo width → distance         | HC-SR04 datasheet (community-published; no ST equivalent): 10 $\mu$ s trigger, echo width, the /58 cm constant                                          |

## L10 — Analog-to-Digital Conversion

The lecture builds counts volts fluency on one convention ( $mV = raw \times 3300 / 4095$ ), one clock ( $PCLK2 \div 2 = 8$  MHz), and one converter (the SAR core).

| Concept as taught                                             | Where the full story is                                                                                                                                                                                                                      |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The ADC itself</b> — 12-bit SAR, one converter on the F401 | <a href="#">RM0368 Rev 6, §11 <i>ADC</i></a> — p. 213                                                                                                                                                                                        |
| <b>The ADC clock</b> — $PCLK2 \div 2 = 8$ MHz, 36 MHz ceiling | <a href="#">§11.3.2 <i>ADC clock</i></a> — p. 215                                                                                                                                                                                            |
| <b>Channels</b> — 16 external doors + the internal sensors    | <a href="#">§11.3.3 <i>Channel selection</i></a> — p. 215; <a href="#">§11.9 <i>Temperature sensor</i></a> — p. 226                                                                                                                          |
| <b>Single vs continuous conversion</b>                        | <a href="#">§11.3.4–11.3.5</a> — p. 216; what the lecture skips: injected channels, the analog watchdog, external triggers ( <a href="#">§11.6</a> — p. 222)                                                                                 |
| <b>Sampling time and source impedance</b>                     | <a href="#">§11.5 <i>Channel-wise programmable sampling time</i></a> — p. 222; the hard electrical limits (max source resistance vs sampling time) are in the STM32F401RE datasheet (DS10086), <a href="#">ADC characteristics</a>           |
| <b>ADC registers</b>                                          | <a href="#">§11.12</a> — p. 229: <a href="#">ADC_SR</a> — p. 229, <a href="#">ADC_CR1</a> — p. 230, <a href="#">ADC_CR2</a> — p. 232, <a href="#">ADC_SMPR2</a> — p. 234, <a href="#">ADC_SQR3</a> — p. 237, <a href="#">ADC_DR</a> — p. 239 |

| Concept as taught                                        | Where the full story is                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The analog pin mode</b> — MODER = 11                  | <a href="#">RM0368 Rev 6, GPIOx_MODER</a> — p. 158                                                                                                                                                                                                                                                                        |
| <b>HAL ADC API</b>                                       | <a href="#">UM1725 Rev 8, ADC generic driver</a> — p. 56: <a href="#">HAL_ADC_Start</a> — p. 65, <a href="#">HAL_ADC_PollForConversion</a> / <a href="#">HAL_ADC_Start_IT</a> — p. 66, <a href="#">HAL_ADC_GetValue</a> / <a href="#">HAL_ADC_ConvCpltCallback</a> — p. 68, <a href="#">HAL_ADC_ConfigChannel</a> — p. 69 |
| <b>Sampling theory and aliasing</b> — stated, not proven | <a href="#">Lee &amp; Seshia</a> , sensors/actuators and sampling chapters; <a href="#">White 2024</a> , analog-input chapters                                                                                                                                                                                            |

## L11 — The Bridge to Assembly

Every listing in the deck is real output of `arm-none-eabi-gcc 14.3` — the toolchain inside STM32CubeCLT — compiled for Cortex-M4 at `-O0` and `-O2`.

| Concept as taught                                                     | Where the full story is                                                                                                                                                                           |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The register file</b> — R0–R12, SP, LR, PC, xPSR                   | <a href="#">PM0214 Rev 10, §2.1.3 <i>Core registers</i></a> — p. 18                                                                                                                               |
| <b>Thumb-2</b> — 16- and 32-bit encodings, the <code>.w</code> suffix | <a href="#">PM0214 Rev 10, §3.1 <i>Instruction set summary</i></a> — p. 50; <a href="#">§3.3.8 <i>Instruction width selection</i></a> — p. 68                                                     |
| <b>Memory access</b> — LDR/STR, offsets, PUSH/POP                     | <a href="#">§3.4</a> — p. 69; <a href="#">§3.4.2 <i>immediate offset</i></a> — p. 71; <a href="#">§3.4.7 <i>PUSH and POP</i></a> — p. 78                                                          |
| <b>Data processing</b> — arithmetic, logic, shifts, CMP, MOV          | <a href="#">§3.5</a> — p. 81: <a href="#">ADD/SUB</a> — p. 83, <a href="#">AND/ORR/EOR/BIC</a> — p. 85, shifts — p. 62, <a href="#">CMP</a> — p. 88, <a href="#">MOV/MVN</a> — p. 89              |
| <b>Branches and conditions</b> — B/BL/BX, condition codes, IT         | <a href="#">B, BL, BX, BLX</a> — p. 142; <a href="#">§3.3.7 <i>Conditional execution</i></a> — p. 65; the <code>-O2</code> ITE surprise: <a href="#">IT</a> — p. 145                              |
| <b>Calling convention</b> — args in r0–r3, result in r0, LR           | The Arm AAPCS32 ( <i>Procedure Call Standard for the Arm Architecture</i> ) — search “AAPCS32” at <a href="#">developer.arm.com</a> ; the lecture teaches only the subset visible in the debugger |
| <b>Interrupt entry stacking</b> — same stack, hardware-driven         | the L06 rows above (exception model); the deck only points back to them                                                                                                                           |

| Concept as taught                                                        | Where the full story is                                                                                                   |
|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>What the optimizer may do</b> —<br>folding, inlining, volatile limits | <a href="#">GCC manual</a> , <i>Optimize Options</i> ;<br><a href="#">White 2024</a> , debugging/optimization<br>chapters |

*Sections for other lectures are added as each lecture is revised to the 2026–2027 pattern.*